

UNIVERSITÉ LIBRE DE BRUXELLES

Langages de programmation 2 – INFO-F-202

Projet Arkanoid - Rapport



Auteurs : Jawad Cherkaoui - Rayan Rabeh

Année académique 2024-2025

Table des matières

1	Introduction	2
2	Tâches réalisées	2
2.1	Tâches de base	2
2.2	Tâches additionnelles implémentées	2
3	Description des classes	3
3.1	Classe Engine (moteur de jeu)	3
3.2	Classe GameModel (modèle du jeu)	3
3.3	Classe GameController (contrôleur du jeu)	4
3.4	Classe GameView (vue du jeu)	5
3.5	Classe Spaceship (raquette du joueur)	6
3.6	Classe Ball (balle du jeu)	7
3.7	Classe Brick (brique du jeu)	9
3.8	Classe Bonus et sous-classes de Bonus	10
3.9	Autres classes utilitaire	12
4	Logique du jeu	13
5	Utilisation du modèle Modèle-Vue-Contrôleur	16

1 Introduction

Ce projet consiste en la réalisation d'un jeu vidéo inspiré d'Arkanoid (Taito, 1986), développé en C++ avec la bibliothèque Allegro. Le joueur contrôle une raquette pour renvoyer une balle et détruire des briques, avec pour objectif de toutes les éliminer sans perdre ses vies. Le développement a permis de mettre en pratique les principes de la programmation orientée objet au sein d'une architecture Modèle-Vue-Contrôleur (MVC).

Le rapport présente les fonctionnalités implémentées, la structure des classes principales, la logique de jeu ainsi que l'application du modèle MVC dans la conception

2 Tâches réalisées

2.1 Tâches de base

Toutes les tâches de base ont été accomplies : le déplacement de la raquette au clavier et à la souris, la gestion de la trajectoire de la balle en fonction de la position de rebond, la mise en place d'une grille de 8 lignes et 14 briques, l'affichage et l'incrémentation du score, la gestion des trois vies avec affichage et message de défaite en cas de perte, ainsi que l'affichage de la victoire lorsque toutes les briques sont détruites.

2.2 Tâches additionnelles implémentées

Du côté des tâches additionnelles, tout a été implémenté à l'exception du bonus Interruption (Split) et du bonus Laser. Le projet intègre donc : la gestion de plusieurs niveaux encodés en fichiers JSON avec raccourcis clavier pour naviguer entre eux, le contrôle de la raquette par la souris, l'ajout de briques colorées attribuant des points différents, la sauvegarde et la réinitialisation du meilleur score, les briques argentées (nécessitant deux coups) et dorées (indestructibles), ainsi que les bonus Agrandir, Attraper, Ralentissement et Vie supplémentaire.

L'ensemble de ces fonctionnalités a été intégré et testé, garantissant un jeu complet et fidèle aux objectifs fixés.

3 Description des classes

L'architecture du projet est organisée selon le modèle Modèle-Vue-Contrôleur, avec des classes réparties par responsabilité. Chaque classe principale est présentée ci-dessous avec son interface publique (les méthodes essentielles) et une description de son rôle et de ses relations avec les autres classes.

3.1 Classe Engine (moteur de jeu)

```
class Engine {
public:
    Engine();
    ~Engine();
    void run();
    // ...
};
```

Rôle

La classe `Engine` est le moteur principal du jeu. Elle initialise la configuration grâce à `AllegroConfig` et `UIConfig`, gère la boucle principale qui enchaîne les niveaux, les menus et la fin de partie, et centralise les événements du clavier, de la souris et du minuteur via `AllegroInputAdapter`. Elle crée et pilote un `GameController` pour la logique du jeu, interagit avec la `GameView` pour l'affichage et les sons, et utilise le `GameModel` pour gérer l'état des niveaux ainsi que la détection de la fin de partie.

3.2 Classe GameModel (modèle du jeu)

```
class GameModel {
public:
    GameModel(const string& levelFile, int score);
    void checkCollisions();
    void updateBonuses();
    void updateTimerBonuses();
    // Getters (score, highScore, ball, spaceship, bricks,
    bonuses)
```

```

    bool win() const;
    bool lose() const;
    // ...
};

```

Rôle

La classe `GameModel` représente le modèle du jeu et concentre tout l'état ainsi que la logique de progression. Elle maintient la balle, la raquette du joueur, la liste des briques du niveau courant, les bonus actifs, le score et le meilleur score. Lors de son initialisation, elle charge les données du niveau à partir d'un fichier et crée les objets `Ball`, `Spaceship` et la collection de `Brick` correspondante. Elle gère aussi la logique des collisions et des rebonds, la mise à jour du score, l'apparition et la gestion des bonus, ainsi que les conditions de victoire et de défaite. `GameModel` n'interagit pas directement avec l'affichage : il se limite à exposer l'état du jeu, qui est exploité par le `GameController` pour relier la logique au rendu et aux entrées utilisateur.

3.3 Classe GameController (contrôleur du jeu)

```

class GameController {
public:
    GameController(const shared_ptr<GameModel>& model,
                  const shared_ptr<GameView>& view,
                  const vector<string>& levelFiles);
    void handleAction(const InputAction& action);
    void update();
    // Getters: getModel(), getView(), getIndex(),
    //           getGameScore()
    // ... (setters for score/index reset/increment)
};

```

Rôle

La classe `GameController` joue le rôle de contrôleur dans l'architecture MVC en reliant les entrées utilisateur au modèle et à la vue. Elle maintient une référence vers le `GameModel` courant et la `GameView`, et traduit les actions reçues (par exemple déplacer la raquette, lancer la balle, réinitialiser un niveau ou passer de force au suivant) en appels appropriés sur le modèle. À chaque tick du jeu, sa méthode `update()` coordonne l'évolution de la partie : elle demande au modèle de vérifier les collisions et de mettre à jour les bonus et leurs effets, contrôle la perte éventuelle d'une balle et la gestion des vies, détecte les changements de direction pour déclencher les sons correspondants, et met à jour la position de la balle en fonction de son état (lancée ou encore fixée à la raquette).

Une fois l'état du modèle mis à jour, le contrôleur ordonne à la vue de rafraîchir l'affichage et de jouer les effets sonores nécessaires. Enfin, il gère aussi la progression entre les niveaux et l'accumulation du score global. En résumé, `GameController` est l'intermédiaire qui synchronise actions utilisateur, logique de jeu et rendu graphique/sonore.

3.4 Classe GameView (vue du jeu)

```
class GameView {
public:
    GameView(const shared_ptr<GameModel>& model);
    void render();
    void menu(ALLEGRO_BITMAP* image, const string& text,
        ...);
    void menuButton(ALLEGRO_BITMAP* image, bool& resultFlag
        , ...);
    void playSound(ALLEGRO_SAMPLE* sample,
        ALLEGRO_SAMPLE_ID* id = nullptr, bool loop=false);
    void stopSound(ALLEGRO_SAMPLE_ID* id);
    UIConfig* getUIConfig();
    AllegroConfig* getAllegroConfig();
    // ...
};
```

Rôle

La classe `GameView` correspond à la vue du jeu : elle s'occupe exclusivement de l'affichage graphique et de la restitution sonore. Reliée au `GameModel`, elle consulte l'état courant de la partie (positions de la balle, de la raquette, des briques, score, vies et bonus) et, via sa méthode `render()`, dessine à chaque frame la scène complète sans jamais modifier le modèle. Elle fournit aussi des méthodes dédiées à l'affichage des menus et écrans spéciaux (par exemple, écran de victoire, défaite ou menu principal), pouvant intégrer images, textes, boutons interactifs et musiques de fond. C'est également elle qui encapsule la gestion audio en jouant ou en arrêtant les sons et musiques, sur demande du contrôleur ou de l'`Engine`.

Enfin, la vue met à disposition la configuration de l'interface et les ressources Allegro (polices, images, sons, fenêtre, événements) de manière centralisée, afin que moteur et contrôleur puissent s'appuyer sur les mêmes éléments. En résumé, `GameView` est responsable de tout ce qui est visuel et sonore, se contentant de refléter fidèlement l'état du modèle tel qu'orchestré par le contrôleur.

3.5 Classe Spaceship (raquette du joueur)

```
class Spaceship : public Rectangle {
public:
    Spaceship(const Point& position, float w, float h,
              float speed, int health);
    void move(Direction dir);
    void setX(float x);
    void damage();           // perd une vie
    void addLife();          // gagne une vie
    void setWidth(float w);
    // Getters: getWidth(), getHeight(), ...
    void setExpandTimer(float t);
    void updateExpandTimer();
    bool isLaserActive() const;
    void setLaser(bool active);
    // ...
};
```

Rôle

La classe `Spaceship` représente la raquette contrôlée par le joueur. Héritant de la classe utilitaire `Rectangle`, elle dispose des propriétés géométriques nécessaires pour détecter les collisions avec la balle et les bonus, tout en se déplaçant uniquement de manière horizontale au bas de l'écran. Elle gère sa position, sa largeur et sa hauteur, sa vitesse de déplacement ainsi que son nombre de vies. Les déplacements sont assurés par la méthode `move()` en fonction d'une direction, ou directement par `setX()` pour suivre la souris.

La raquette prend également en charge certains bonus : `EXPAND` qui modifie temporairement sa largeur jusqu'à expiration d'un timer, `LASER` qui active un mode de tir (prévu mais non implémenté en détail), et `EXTRA_LIFE` qui ajoute une vie. En cas de perte de balle, la méthode `damage()` décrémente son compteur de vies. La raquette n'interagit pas directement avec la balle ni avec les briques ; elle se limite à maintenir son propre état et à fournir au `GameModel` les informations nécessaires pour la gestion des collisions et des rebonds.

3.6 Classe Ball (balle du jeu)

```
class Ball : public Circle {
public:
    Ball(float radius, float speed);
    void move(const Point& spaceshipCenter, float
        spaceshipH);
    void move(const Point& spaceshipCenter, float
        spaceshipW, float spaceshipH);
    void checkCollisions(const Point& spaceshipCenter,
        float spaceshipW, float spaceshipH);
    void checkFall();
    void reset();
    void setMoving(bool moving);
    bool isMoving() const;
    bool isFalling() const;
    void setFalling(bool falling);
    // Getters/Setters: getPosition(), getDirection(),
        setDirection(...),
```



```

//                                getSpeed(), setSpeed(...),
                                getOriginalSpeed(), ...
void setCatchActive(bool active);
void setCatchReleaseTimer(float t);
void updateCatchReleaseTimer();
void updateSpeed();
// ...
};

```

Rôle

La classe **Ball** représente la balle du casse-briques et hérite de **Circle** pour gérer sa position, son rayon et ses collisions circulaires de base. Elle maintient son vecteur de direction et sa vitesse, ainsi que plusieurs indicateurs d'état : **moving** pour savoir si elle est lancée ou attachée à la raquette, **falling** pour signaler une perte de balle, et **catchActive** lorsqu'elle doit rester collée à la raquette sous l'effet du bonus CATCH. Elle se déplace soit en suivant la raquette lorsqu'elle est attachée, soit en avançant selon son vecteur de direction lorsqu'elle est en mouvement, en vérifiant à chaque tick ses collisions avec les murs et la raquette.

Lors d'un rebond sur la raquette, l'angle de réflexion est ajusté en fonction du point d'impact pour diversifier la trajectoire. La balle détecte aussi si elle sort par le bas de l'écran et déclenche alors l'état de chute, ce qui signale au contrôleur de décrémenter une vie et de la réinitialiser sur la raquette. Elle propose des méthodes pour être relancée, réinitialisée, marquée en chute ou placée en mode CATCH, ce dernier étant géré par un timer de relâche automatique. Enfin, la balle prend en charge le retour progressif à sa vitesse initiale après un ralentissement causé par le bonus SLOW_DOWN.

En résumé, **Ball** encapsule la physique de la balle, ses états et ses effets spéciaux, tandis que la détection des collisions avec les briques reste de la responsabilité du **GameModel**.

3.7 Classe Brick (brique du jeu)

```
class Brick : public Rectangle {
public:
    Brick(const Point& position, float w, float h, int
          score, BonusType bonus = BonusType::NONE);
    void destroy();
    bool isDestroyed() const;
    bool getSecondLife() const;
    void setSecondLife(bool value);
    int  getScore() const;
    BonusType getBonusType() const;
    // ...
};
```

Rôle

La classe `Brick` représente une brique du casse-briques et hérite de `Rectangle` afin de bénéficier des fonctions de collision nécessaires pour détecter les impacts avec la balle. Chaque brique possède un score, un type de bonus éventuel, ainsi que deux indicateurs d'état : `destroyed` pour savoir si elle est détruite et `secondLife` pour gérer les briques renforcées qui nécessitent deux coups avant d'être éliminées. Le constructeur initialise ces valeurs selon les paramètres (par exemple, une brique renforcée est créée avec `secondLife=true`, une brique indestructible peut être associée au score `GOLD_BRICK`).

Ses méthodes sont simples et limitées à la gestion de son état : `destroy()` marque la brique comme détruite, `setSecondLife(false)` supprime la résistance d'une brique renforcée après le premier impact, et des getters permettent d'accéder à ses attributs (score, bonus, état détruit ou non, seconde vie). La classe elle-même reste passive : elle ne prend aucune décision et n'interagit pas directement avec la balle ou les bonus.

C'est le `GameModel` qui orchestre la logique de destruction, l'attribution des points, l'éventuelle création d'un bonus, et qui décide de la façon dont l'état de la brique influe sur le déroulement du jeu.

3.8 Classe Bonus et sous-classes de Bonus

```
enum class BonusType { NONE, LASER, EXPAND, CATCH,
    SLOW_DOWN, SPLIT, EXTRA_LIFE };

class Bonus : public Rectangle {
public:
    Bonus(const Point& pos, float w, float h, BonusType
        type, float fallSpeed);
    void update();
    void applyEffect(GameModel& model);
    bool isActive() const;
    bool isCollected() const;
    BonusType getType() const;
    // ...
};

class Expand      : public Bonus { public: void applyEffect
    (GameModel& m) override; /*...*/ };
class Catch       : public Bonus { public: void applyEffect
    (GameModel& m) override; /*...*/ };
class Laser       : public Bonus { public: void applyEffect
    (GameModel& m) override; /*...*/ };
class SlowDown    : public Bonus { public: void applyEffect
    (GameModel& m) override; /*...*/ };
class Split       : public Bonus { public: void applyEffect
    (GameModel& m) override; /*...*/ };
class ExtraLife   : public Bonus { public: void applyEffect
    (GameModel& m) override; /*...*/ };
```

Rôle

La classe `Bonus` est une classe abstraite qui représente un bonus libéré par une brique et pouvant être ramassé par la raquette. Elle hérite de `Rectangle` afin d'être détectée comme un petit objet rectangulaire tombant verticalement et d'interagir avec la raquette par collision. Chaque bonus possède un type, une vitesse de chute et deux états : `active` pour indiquer qu'il est en train de tomber et `collected` pour signaler qu'il a été ramassé. Sa méthode `update()` gère simplement la descente en mettant à jour la position en fonction de la vitesse.

Les effets sont définis dans les sous-classes, chacune redéfinissant `applyEffect(GameModel&)`. Ainsi, `Expand` agrandit la raquette et active un timer, `Catch` permet à la balle de se coller à la raquette, `Laser` active le mode laser sur la raquette, `SlowDown` ralentit fortement la balle avant qu'elle ne retrouve sa vitesse normale, `Split` devait dupliquer la balle mais n'est pas réellement implémenté, et `ExtraLife` ajoute une vie supplémentaire au joueur.

Le `GameModel` gère l'ensemble du cycle de vie des bonus : il les instancie depuis les données de niveau, les fait tomber à chaque frame, détecte leur collision avec la raquette et applique leur effet en appelant la méthode adéquate. Il conserve aussi en mémoire le bonus actuellement actif pour pouvoir l'annuler ou le remplacer par un nouveau (par exemple rétrécir la raquette après la fin d'un bonus `EXPAND`).

En somme, `Bonus` fournit une structure générique pour représenter et faire tomber les bonus, tandis que la logique propre à chaque effet est encapsulée dans les sous-classes spécialisées, et que le `GameModel` centralise leur gestion et leur durée de vie.

3.9 Autres classes utilitaire

- **InputAction** / **InputActionType** : énumère les actions utilisateur (clavier, souris, timer, etc.) et associe des données (ex. coordonnées souris). Produites par l'adaptateur d'entrée, consommées par le **GameController**.
- **AllegroInputAdapter** : convertit les événements bruts Allegro (clavier, souris, timer, fenêtre) en **InputAction** abstraites (ex. touche gauche → **MoveLeft**, timer → **TimerTick**).
- **AllegroConfig** / **UIConfig** : regroupent les ressources et paramètres du jeu.
 - **AllegroConfig** : bas niveau (display, event queue, timer, images, sons).
 - **UIConfig** : interface (dimensions briques, positions UI, boutons, etc.).
- **Shapes** (Circle, Rectangle) : primitives géométriques et collisions AABB. Ball hérite de Circle, Spaceship/Brick de Rectangle.
- **Vector** (Vector.hpp) : utilitaire géométrique pour optimiser la détection de collisions (points d'impact, côtés touchés, rebonds précis).

4 Logique du jeu

(du lancement au premier impact de brique)

Au lancement, `main()` construit `Engine` puis appelle `Engine::run()`. Le constructeur d'`Engine` initialise `AllegroConfig` (création de la fenêtre, event queue, timer 60 FPS, chargement des bitmaps/samples) et `UIConfig` (polices, chemins d'assets), recense la liste des niveaux (par ex. via un scan de `assets/data/level_*.json`) et lève immédiatement une exception si la liste est vide. L'invariant à ce stade est : « config OK, niveaux non vides, event queue et timer prêts ».

L'entrée dans `Engine::run()` déclenche l'écran d'accueil via `handleStartMenu()` qui, à l'aide de `GameView::menuButton()`, affiche un fond, du texte et deux boutons Play/Quit. Tant que l'utilisateur n'a pas choisi, la boucle d'attente consomme les événements Allegro et ne progresse pas dans le jeu ; au clic sur Quit, `handleStartMenu()` retourne `false` et `run()` termine ; au clic sur Play, elle retourne `true`, `showMenu` passe à `false` (invariant : on n'affichera plus ce menu pour ce run) et l'index de niveau courant est fixé à 0.

L'`Engine` instancie alors le triplet MVC pour le niveau courant : `GameModel` reçoit le chemin du JSON de niveau et, dans son initialisation, crée la grille de `Brick` (positions mondes et métadonnées score/bonus/second life), initialise `Spaceship` (position X centrée en bas, largeur/hauteur selon `settings.json`, `health` initial), et place `Ball` juste au-dessus de la raquette, centrée horizontalement, avec `moving=false`, `falling=false`, une direction initiale normalisée vers le haut `INITIAL_DIRECTION` et une `speed` nominale. L'invariant critique côté modèle est : « aucun bonus actif, score=0, toutes les briques non détruites, balle attachée à la raquette ». `GameView` est construit avec une référence vers le modèle et prépare les ressources d'affichage/son nécessaires ; `GameController` est construit avec (modèle, vue, liste de niveaux) et réinitialise ses états internes (par ex. `tempDirection` ← direction initiale de la balle). On entre alors dans `Engine::runGameLevel()` : le timer (60 FPS) est démarré, une musique de fond optionnelle est lancée via `GameView::playSound(..., loop=true)` et l'`Engine` commence une boucle `while (running)` qui bloque sur `al_wait_for_event(queue,`

`&ev`); chaque événement brut est traduit par `AllegroInputAdapter` en `InputAction` (par exemple `TimerTick`, `MoveLeft`, `MoveRight`, `MouseMove`, `Launch`, `Reset`, `Quit`, `LevelChange`).

Pour chaque action, `Engine::processAction(action)` s'exécute : si `Quit`, on sort proprement (arrêt musique, retour `false`) ; si `TimerTick`, on appelle `controller.update()` puis `viewrender()` ; si c'est une action de contrôle, on appelle `controller.handleAction(action)` (effet immédiat sur le modèle, rendu au tick suivant). Après traitement, l'`Engine` vérifie les conditions terminales via `GameModel::win()` et `lose()` ; en cas de victoire, la musique est stoppée, un écran de victoire est joué ; en cas de défaite, un écran rejouer/quitter apparaît et le score global peut être réinitialisé. Tant que rien n'est terminal, la boucle continue.

Dans l'état initial du niveau (balle attachée), tant que `Launch` n'a pas été reçu, `controller.update()` aligne la balle sur la raquette par la surcharge `Ball::move(spaceshipCenter, spaceshipH)` : la balle est rendue collée, `Ball::checkCollisions(...)` ne déclenche rien (pas de mur, pas de raquette — on colle au-dessus), et la direction n'est pas modifiée. Dès que le joueur presse Espace, l'événement clavier est converti en `InputActionLaunch` ; `controller.handleAction(Launch)` appelle `ball.setMoving(true)` et ne touche ni à la position ni à la direction (invariant : la direction reste `INITIAL_DIRECTION` vers le haut ; cela garantit un lancement déterministe). Le tick suivant (provenant du timer) appelle `controller.update()` ;

L'ordre interne strict est : (1) `model.checkCollisions()` sur la scène bricks/balle pour ce pas de temps (ray-cast ou balayage discret le long du vecteur de déplacement théorique du tick), (2) `model.updateBonuses()` et `updateTimerBonuses()` (à ce stade, aucun effet), (3) détection d'un changement de direction par comparaison `tempDirection` vs direction courante via `model.checkDirectionChanged(tempDirection)` — ici attendu `false` car pas de rebond encore —, puis (4) déplacement effectif de la balle via la surcharge `Ball::move(spaceshipCenter, spaceshipW, spaceshipH)` puisque `ball.isMoving()==true`. Ce déplacement applique la cinématique `pos += dir * speed * dt_tick` avec normalisation de `dir` garantie, ap-

pelle `Ball::checkCollisions(...)` pour murs/raquette uniquement, et, dans cette phase initiale, ne rencontre ni mur ni raquette (on vient de quitter la raquette et on est loin du plafond), donc la direction reste inchangée.

À la fin de `update()`, `tempDirection` est synchronisé sur la direction actuelle pour le tick suivant (invariant de cohérence sonore : “un seul son par changement de direction effectif”). `viewrender()` lit l’état du modèle et dessine : briques intactes, raquette, balle un peu plus haut, score encore nul.

Ce cycle `TimerTick` → `update` → `render` se répète ; la balle progresse verticalement vers la zone de briques. Dès que, pour un tick donné, la distance projetée de déplacement de la balle intercepte la première brique de la grille dans `dir` (le modèle la trouve par ray-cast ou par itération spatiale ordonnée, peu importe, l’invariant est « la première brique atteinte le long du segment de déplacement de ce tick est sélectionnée de façon déterministe »), `model.checkCollisions()` retourne un `CollisionResult` contenant un pointeur vers la `Brick` touchée et un `hitSide` précis (face haute/basse/-gauche/droite, coin possible).

Le modèle appelle alors `handleBrickHit(brick)` : si la brique est indestructible (`score==GOLD_BRICK`) elle est laissée intacte ; si `secondLife==true`, la résistance est consommée via `brick.setSecondLife(false)` sans score ni destruction ; sinon, `brick.destroy()` est appelée, `score += brick.score()` est cumulé, et un bonus éventuel est spawné (`bonusType!=NONE`) et ajouté à `model.bonuses`. Immédiatement après, `handleBallRebound(direction, hitSide)` calcule la nouvelle direction en inversant la composante adéquate (y pour un impact sur face horizontale, x pour une face verticale ; double inversion sur coin), avec clamp d’angle si nécessaire (bornes `BOUNCE_MIN_DEG/MAX` pour éviter les trajectoires dégénérées).

L’invariant post-collision est : « la brique a un état cohérent (détruite ou pas), le score est à jour, la direction de la balle est mise à jour avant tout rendu ». `model.updateBonuses()` et `updateTimerBonuses()` s’exécutent ensuite ; juste après une destruction sans bonus il n’y a rien à faire, sinon un nouvel objet `Bonus` actif descend déjà à la frame suivante. Le

contrôleur compare alors `tempDirection` à la nouvelle direction ; le changement étant effectif, un son de rebond/casse est déclenché par la vue, puis `Ball::move(...)` avance la balle selon la direction post-rebond pour la portion de déplacement de ce tick.

Finalement, `viewrender()` peint la scène avec la brique désormais non dessinée (ou dessinée en état « cassée » selon choix), la balle proche du point d'impact et le score mis à jour ; si un bonus a été spawné, il apparaît à la position de l'ancienne brique avec `active=true`.

Conclusion : le chemin exact du lancement au premier impact est donc

`main` → `Engine::run` → `handleStartMenu` → (MVC init) → `runGameLevel` → (`TimerTick/Inputs`) jusqu'à `controller.handleAction(Launch)` → `controller.update()` où,

au premier tick de collision, `model.checkCollisions()` sélectionne la première brique interceptée, `handleBrickHit` met à jour score/état/bonus, `handleBallRebound` ajuste la direction avec bornes d'angles, `checkDirectionChanged` déclenche le son, `Ball::move` finalise la position de fin de tick, puis `viewrender` reflète visuellement l'ensemble — garantissant des invariants de cohérence (un seul rebond par tick côté modèle, direction normalisée, score monotone, état des briques idempotent).

5 Utilisation du modèle Modèle-Vue-Contrôleur

Le projet applique clairement le modèle MVC. Le **modèle** (`GameModel`, `Ball`, `Spaceship`, `Brick`, `Bonus`) contient uniquement l'état et la logique du jeu sans dépendre de l'affichage ou des entrées. La **vue** (`GameView`, `AllegroConfig`, `UIConfig`) se limite au rendu graphique et sonore en lisant l'état du modèle, sans jamais le modifier. Le **contrôleur** (`GameController`, orchestré par `Engine`) relie les entrées utilisateur au modèle, déclenche les mises à jour et demande à la vue de refléter les changements. Cette séparation stricte garantit modularité et clarté du code.